

CYBERSECURITY AND THREAT DETECTION SYSTEM

Module-wise Technical Documentation

Capstone Project Report

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Computer Applications (B.C.A.)

Academic Year 2024 - 2025

TABLE OF CONTENTS

1. Module 1 — User Authentication	3
2. Module 2 — Log Collection	5
3. Module 3 — Threat Detection	7
4. Module 4 — Alert and Notification	9
5. Module 5 — Reporting and Dashboard	11

Project Overview

This document presents the complete technical and functional documentation for the Cybersecurity and Threat Detection System, developed as a capstone project for the Bachelor of Computer Applications programme. The system is designed to simulate a real-world, end-to-end security operations pipeline that monitors network activity, detects anomalous behaviour, and communicates alerts to responsible stakeholders.

The system is divided into five functional modules that operate in a sequential pipeline:

1. User Authentication — Verifies and controls access to the security platform.
2. Log Collection — Gathers and structures raw event data from multiple sources.
3. Threat Detection — Analyses collected logs to identify malicious activity.
4. Alert and Notification — Communicates detected threats to administrators.
5. Reporting and Dashboard — Presents analytical summaries and visual insights.

Each module is documented with its purpose, functional requirements, technical design, workflow, and data flow. Together, these five modules form a cohesive, production-inspired cybersecurity platform suitable for academic demonstration and further development.

Module 1: User Authentication

1.1 Overview

The User Authentication module is the entry point and security gatekeeper of the entire system. Its primary function is to verify the identity of users attempting to access the cybersecurity platform and to enforce role-based access controls. No other module is accessible unless this layer has successfully validated the user's credentials.

In a real-world security operations environment, the authentication layer is critical because the platform itself contains sensitive information about vulnerabilities, threat intelligence, and organisational network behaviour. Unauthorised access to such a system would represent a serious security breach in its own right.

1.2 Objectives

- Authenticate users securely using username and password credentials.
- Support multiple roles such as Administrator, Analyst, and Viewer with distinct permission levels.
- Protect against brute-force attacks through account lockout and rate limiting.
- Maintain a session management mechanism that invalidates stale or hijacked sessions.
- Log all authentication events, including both successful logins and failed attempts.

1.3 Functional Requirements

- **Registration:** New users are registered by an administrator. Self-registration is disabled to prevent unauthorised accounts.
- **Login:** Users submit a username and password. The system validates credentials against a hashed password store.
- **Session Token:** Upon successful login, the system generates a secure JSON Web Token (JWT) or session cookie with a defined expiry time.
- **Role Assignment:** Each user is assigned a role at the time of account creation. Roles determine which modules and data are visible.
- **Logout:** Users can manually terminate their session. Tokens are invalidated on logout.
- **Account Lockout:** After five consecutive failed login attempts, the account is temporarily locked for thirty minutes.

1.4 Technical Design

Technology Stack

- Backend: Python (Flask or Django REST Framework)

- Password Hashing: bcrypt with salt rounds set to a minimum of 12
- Session Management: JWT tokens with RS256 signing algorithm
- Database: MySQL or PostgreSQL for user records
- Frontend: HTML5, CSS3, JavaScript (Vanilla or React)

Authentication Flow

6. User submits credentials via the login form.
7. The server retrieves the hashed password associated with the provided username.
8. bcrypt compares the submitted password against the stored hash.
9. If valid, a signed JWT is issued and returned to the client.
10. Subsequent requests include the JWT in the Authorization header.
11. The server validates the token's signature and expiry on every protected route.

1.5 Security Considerations

- Passwords are never stored in plaintext or reversibly encrypted form.
- All communication occurs over HTTPS to prevent credential interception.
- JWTs contain no sensitive user data beyond the user ID and role.
- Refresh tokens are stored server-side and can be revoked independently of the access token.

1.6 Output to Next Module

Upon successful authentication, the module passes the verified user identity, role, and session token to the application context. Authenticated analyst and administrator users are granted access to the Log Collection module.

Module 2: Log Collection

2.1 Overview

The Log Collection module is the data ingestion layer of the system. It is responsible for gathering raw event data from a variety of simulated or real network sources, normalising that data into a consistent format, and storing it in a centralised log repository for downstream analysis. Without a reliable and well-structured log pipeline, the Threat Detection module cannot operate effectively.

In enterprise environments, log collection is handled by tools such as Splunk, Elastic Stack (ELK), or Fluentd. This module replicates the core principles of such systems at an academic scale.

2.2 Objectives

- Collect event logs from simulated sources including web servers, firewalls, and operating systems.
- Parse and normalise raw log entries into a standard schema.
- Filter out duplicate or irrelevant events to reduce noise.
- Store structured logs in a queryable database.
- Provide a real-time feed of incoming log entries to the detection engine.

2.3 Supported Log Sources

- **Web Server Logs:** Apache or Nginx access and error logs containing HTTP requests, response codes, and client IP addresses.
- **Firewall Logs:** Simulated firewall deny and permit records showing source IP, destination IP, port, and protocol.
- **System Logs (Syslog):** Operating system event messages indicating user logins, sudo usage, and service status changes.
- **Authentication Logs:** Logs forwarded from Module 1 capturing login successes, failures, and lockout events.

2.4 Log Normalisation Schema

All incoming log entries, regardless of source format, are normalised into the following unified structure before storage:

- **timestamp:** ISO 8601 format date and time of the event.
- **source_ip:** IP address of the originating host.
- **destination_ip:** IP address of the target host, if applicable.

- **event_type:** Category such as AUTH_FAILURE, HTTP_REQUEST, or PORT_SCAN.
- **severity:** Enumerated value: INFO, WARNING, ERROR, or CRITICAL.
- **raw_message:** The original unmodified log line preserved for audit purposes.
- **source_module:** Identifier indicating which log source produced this entry.

2.5 Technical Design

- Log ingestion uses a scheduled polling agent or a real-time file watcher (Python watchdog library).
- Regex-based parsers are written for each log source format.
- Parsed entries are inserted into a relational database table: logs_raw.
- A background worker process continuously moves normalised logs to logs_normalised.
- Indexing is applied on timestamp and source_ip columns to enable fast querying.

2.6 Output to Next Module

The normalised log entries stored in logs_normalised serve as the primary input to the Threat Detection module. The detection engine queries this table in real time, applying rule-based and pattern-matching algorithms to identify suspicious activity.

Module 3: Threat Detection

3.1 Overview

The Threat Detection module is the analytical core of the system. It continuously evaluates the normalised log stream produced by the Log Collection module and applies a combination of signature-based detection rules and behavioural heuristics to identify events that indicate a potential security threat. Detected threats are passed to the Alert and Notification module for escalation.

This module draws inspiration from Intrusion Detection Systems (IDS) such as Snort and Suricata, as well as the detection logic used in commercial Security Information and Event Management (SIEM) platforms.

3.2 Objectives

- Continuously monitor the normalised log stream for suspicious patterns.
- Apply predefined rules to detect known attack signatures.
- Use behavioural analysis to flag anomalous activity that does not match known patterns.
- Assign a threat severity score to each detected incident.
- Avoid false positives by incorporating a configurable confidence threshold.

3.3 Detection Techniques

Signature-Based Detection

Signature-based detection compares incoming log entries against a library of known attack patterns. When a log entry matches a rule, a threat event is generated immediately. Examples of signature rules implemented in this system include:

- More than ten failed login attempts from the same IP address within five minutes — classified as a Brute Force Attack.
- HTTP requests containing SQL injection payloads such as `OR 1=1` or `UNION SELECT` — classified as an SQL Injection Attempt.
- Rapid sequential connection attempts to more than twenty distinct ports — classified as a Port Scan.
- HTTP 404 responses exceeding fifty within sixty seconds from one source — classified as a Directory Traversal Attempt.

Anomaly-Based Detection

Anomaly-based detection establishes a baseline of normal behaviour over a defined observation period and flags deviations beyond a statistical threshold. Metrics monitored include:

- Unusual login times for known user accounts.
- Data transfer volumes significantly higher than historical averages.
- Access to resources not previously accessed by a given user role.

3.4 Threat Severity Classification

- **Critical:** Active exploitation confirmed or imminent. Requires immediate response.
- **High:** Strong indicator of attack intent. Escalation recommended within fifteen minutes.
- **Medium:** Suspicious activity warranting investigation. Not immediately harmful.
- **Low:** Informational event. Logged but does not trigger an alert.

3.5 Technical Design

- Detection engine implemented in Python, running as a background daemon.
- Rule definitions stored in a YAML configuration file for easy modification.
- Each detection cycle queries the last sixty seconds of normalised logs.
- Matched threats are written to a `threats_detected` table with full contextual metadata.
- A confidence score between 0.0 and 1.0 is computed for each threat; events below 0.5 are suppressed from alerts.

3.6 Output to Next Module

Each confirmed threat event, including its type, severity, source, timestamp, and confidence score, is forwarded to the Alert and Notification module. The detection engine also writes threat records to the database for historical analysis by the Reporting and Dashboard module.

Module 4: Alert and Notification

4.1 Overview

The Alert and Notification module bridges the gap between automated threat detection and human response. When a confirmed threat event is generated by the Threat Detection module, this module is responsible for constructing a meaningful, actionable alert and delivering it to the appropriate personnel through one or more communication channels in a timely manner.

The effectiveness of any security system depends not only on its ability to detect threats, but also on how quickly and clearly that information reaches the people who can act on it. This module is designed with that urgency in mind.

4.2 Objectives

- Receive confirmed threat events from the Threat Detection module in real time.
- Route alerts to the appropriate recipients based on threat severity and user roles.
- Deliver notifications through multiple channels: in-app, email, and SMS.
- Prevent alert fatigue by deduplicating repeated notifications for the same ongoing event.
- Allow administrators to acknowledge, assign, or dismiss alerts from the dashboard.

4.3 Notification Channels

In-Application Notifications

Real-time alerts are displayed within the dashboard interface using WebSocket-based push notifications. A notification badge counter increments on the navigation bar and a notification panel lists all unacknowledged alerts with their severity, description, and timestamp.

Email Notifications

For High and Critical severity threats, an automated email is dispatched to all users with Administrator or Analyst roles. The email includes the threat type, source IP, affected system, severity level, time of detection, and a direct link to the relevant log entries in the dashboard.

SMS Notifications

Critical severity events trigger an SMS message to a pre-configured mobile number using an API such as Twilio or Fast2SMS. The message is intentionally brief, containing the threat type, severity, and time only, with a prompt to log in for full details.

4.4 Alert Deduplication

To prevent administrators from receiving dozens of identical notifications for a sustained attack such as a prolonged brute-force attempt, the module applies a deduplication window. If the same threat type from the same source IP is detected within a ten-minute window, only one alert is dispatched. A counter within the alert is incremented to indicate the number of occurrences.

4.5 Alert Lifecycle

12. A new threat event is received from the detection engine.
13. The module checks whether a matching active alert exists within the deduplication window.
14. If no duplicate exists, a new alert record is created with status: NEW.
15. Notifications are dispatched to all relevant recipients.
16. An analyst reviews the alert and sets its status to: ACKNOWLEDGED.
17. After investigation, the analyst marks the alert as: RESOLVED or FALSE POSITIVE.
18. All status transitions are timestamped and recorded for audit purposes.

4.6 Output to Next Module

All alert records, including creation timestamps, severity levels, notification history, and resolution status, are written to the alerts table in the database. This data feeds directly into the Reporting and Dashboard module for trend analysis and audit reporting.

Module 5: Reporting and Dashboard

5.1 Overview

The Reporting and Dashboard module is the presentation and analysis layer of the system. It aggregates data from all preceding modules — authentication events, collected logs, detected threats, and alerts — and presents them through an interactive, real-time web dashboard along with exportable periodic reports.

This module transforms raw operational data into strategic intelligence, enabling security teams to understand historical trends, measure system performance, identify repeat offenders, and communicate findings to non-technical stakeholders through structured reports.

5.2 Objectives

- Provide a centralised dashboard view of the system's current security posture.
- Display real-time metrics including active threats, recent alerts, and log ingestion rate.
- Enable historical analysis of threats by time period, type, source, and severity.
- Automatically generate daily and weekly summary reports in PDF or Excel format.
- Allow analysts to create custom queries and filter views for investigation purposes.

5.3 Dashboard Components

Summary Statistics Panel

The top section of the dashboard displays four key metric cards updated every thirty seconds: total logs collected in the last 24 hours, total threats detected in the last 24 hours, number of unresolved active alerts, and current system status (Secure, Warning, or Critical).

Threat Timeline Chart

A time-series line chart displays the volume of detected threats over the past seven days, broken down by severity level. This helps analysts identify patterns such as recurring attacks at specific times of day or escalating activity over a week.

Top Threat Sources Table

A ranked table lists the top ten IP addresses responsible for the most threat events, along with their associated attack types, total event count, and first and last seen timestamps. This is particularly useful for identifying persistent attackers or compromised internal hosts.

Alert Status Distribution

A doughnut or pie chart shows the proportion of alerts currently in each lifecycle state: New, Acknowledged, Resolved, and False Positive. This gives team leads a quick view of analyst workload and response efficiency.

Recent Activity Feed

A scrolling live feed in the bottom section of the dashboard shows the twenty most recent log events and threat detections, updated in real time via WebSocket connection. Entries are colour-coded by severity for rapid visual scanning.

5.4 Reporting

Automated Reports

- Daily Summary Report: Generated automatically at midnight. Includes total events, top threats, and alert resolution statistics for the day.
- Weekly Trend Report: Generated every Sunday. Includes week-over-week comparisons, threat category breakdowns, and analyst performance metrics.

On-Demand Reports

Analysts can generate custom reports by specifying a date range, threat type filter, severity filter, and source IP filter. Reports can be exported in PDF format for formal documentation or in CSV format for further analysis in external tools.

5.5 Technical Design

- Dashboard frontend built with HTML5, CSS3, and Chart.js or ApexCharts for visualisation.
- Backend API endpoints built with Python Flask, serving pre-aggregated query results.
- Database views and stored procedures used to optimise complex aggregate queries.
- PDF report generation handled by the ReportLab or WeasyPrint library.
- WebSocket push updates managed by Flask-SocketIO or Django Channels.

5.6 Access Control

The dashboard content is personalised based on the authenticated user's role. Administrators have access to all data, all user management functions, and report generation. Analysts can view all threat and log data but cannot manage user accounts. Viewers have read-only access to the summary panel and timeline chart only.

Conclusion

This document has presented the complete module-wise documentation for the Cybersecurity and Threat Detection System, covering all five functional modules from initial user authentication through to final reporting and visualisation.

The system demonstrates a practical understanding of modern security operations concepts including identity and access management, event log management, intrusion detection techniques, incident response workflows, and security analytics. Each module has been designed with clear inputs, well-defined processing logic, and structured outputs that feed seamlessly into the next stage of the pipeline.

The modular architecture of the system ensures that individual components can be independently tested, extended, or replaced with more sophisticated implementations as the project matures. The documentation provided here is intended to serve as both a technical reference for the development team and a formal academic submission demonstrating the depth and breadth of the project's scope.

Future enhancements that could be incorporated include integration with machine learning-based anomaly detection models, support for additional log sources such as cloud infrastructure providers, and a mobile application for alert management.